

Stage 1 Report:

Predicting Helsinki City Bike Trip Duration from Time and Weather

CS-C3240 Machine Learning

1. Introduction

Each year, hundreds of thousands of people rent shared city bikes across Helsinki and Espoo. Riders currently have no simple way to estimate, before starting a trip, how long it is likely to take to get to the destination. That information could help them decide, for instance, whether cycling is a reasonable choice given the weather and time of day. This project use machine learning to suggest a solution to the problem: given only the day, time, and weather conditions, we predict how long a city bike trip will last. This report will formalizes the application as a machine learning problem and describes and motivates our modelling choices for Stage 1 of the project.

2. Problem Formulation

We treat this as a supervised regression problem. Each data point represents one completed trip recorded by the Helsinki/Espoo city bike system. We describe a trip using five features (see Table 1) and predict its duration in minutes, a continuous, strictly positive quantity obtained directly from the trip record.

Table 1. Features used to describe each trip.

Feature	Type	Description
Weekday	Categorical	Day the trip started (Mon-Sun)
Hour of day	Categorical (ordinal)	Hour the trip started (0-23)
Month	Categorical	Calendar month (Apr-Oct, the bikes operating season)
Mean temperature	Continuous	Day mean air temperature (°C)
Precipitation	Continuous	Day total precipitation (mm)

The trip data comes from HSL's (Helsinki Region Transport) open Origin-Destination dataset, published monthly and licensed CC BY 4.0. Weather data comes from the Finnish Meteorological Institute's (FMI) open data for the Kaisaniemi observation station in Helsinki and is joined to each trip by its calendar date.

3. Methods

3.1 Dataset and preprocessing

Our combined dataset covers all trips recorded during the bikes' operating seasons from April 2022 to October 2025 (28 monthly files), totaling about 10.1 million individual trips. For each trip we join in that day's temperature and precipitation from the FMI weather file. We remove clearly invalid records before modelling: trips with near-zero distance and duration under a few seconds are "false starts" (a bike undocked and immediately redocked) rather than real rides, and are excluded using the trip's recorded distance.

3.2 Feature selection

We chose the five features above after visualizing trip duration against several candidate variables. Grouping trips by hour of day and by weekday/weekend revealed a clear systematic pattern: shorter, commute-like trips during weekday rush hours, and longer, more variable trips on weekends and around midday. Grouping by daily temperature and precipitation similarly showed the typical trip getting shorter in cold or wet conditions. We deliberately excluded the trip's straight-line distance from the model's features, even though it is present in the raw data: distance is essentially fixed by which two stations a rider selects, which our calendar/weather features cannot explain, so including it would let the model "cheat" rather than

learn a genuine time/weather relationship. We do still use distance during preprocessing, as described in Section 3.1.

3.3 Model (hypothesis space)

For Stage 1, we use linear regression as our hypothesis space: duration is modelled as a linear function of the inputs above (one-hot encoded for the categorical features). We chose this as a first, simple hypothesis because a plausible near-linear story underlies the problem: trip duration is approximately distance divided by riding speed, and riding speed itself should shift only incrementally with weather and time of day (a rider is a little slower in the rain, a little faster during a weekday commute), rather than in some sharply non-linear way. Testing whether this simple, interpretable hypothesis is adequate is a reasonable starting point; Stage 2 will compare it against a non-linear alternative.

3.4 Loss function

We fit the linear model using the Huber loss rather than ordinary squared error. Bike-share duration data contains extreme outliers: a small fraction of trips are bikes that were not properly returned and remain “in use” for many hours. Squared error penalizes large residuals quadratically, so these rare outliers can dominate the fit and distort predictions for ordinary trips. The Huber loss behaves like squared error for small residuals, but switches to a linear penalty for large residuals which the influence of forgotten-bike outliers on the fitted line.

3.5 Training and validation split

We split the data by time rather than randomly: trips from the 2022 and 2023 seasons form our training set, and trips from the 2024 season form our validation set. The 2025 season is set aside entirely and is not used at this stage. We use a time-based split, rather than a random one, because the goal of the model is to generalize to future seasons; a random split would let information from nearby days leak between the training and validation sets (e.g., the same week appearing partly in each), giving an artificially optimistic sense of accuracy. Reserving an entire, later season for validation better reflects how the model would actually be used.

4. Use of Generative AI

We use Generative AI (Claude) during this stage 1 of the project to: (1) brainstorm and narrow down candidate applications and datasets before settling on this topic; (2) explain machine learning concepts and terminology referenced in this report; (3) debug and fix the code of the project, and (4) help structure the report based on our own project decisions, data, and code. All modelling decisions, the accompanying code, and the content of this report were made and reviewed by us.

References

HSL (Helsinki Region Transport). Helsinki and Espoo city bike trip data.
https://hri.fi/data/en_GB/dataset/helsingin-ja-espoon-kaupunkipyorilla-ajatut-matkat (CC BY 4.0).

Finnish Meteorological Institute (FMI). Open data, weather observations.
<https://en.ilmatieteenlaitos.fi/open-data> (CC BY 4.0).

Jung, A. Machine Learning: The Basics. <https://mlbook.cs.aalto.fi>.

Appendix: Code